Universidad Técnica Federico Santa María
Campus Santiago San Joaquín
Bases de Datos Avanzadas

Tarea 3: Neo4j

Informe

Nombre: Sebastián Andrés Santander Martínez
Rol: 202373608-2

Nombre: Jaime Guzmán Muñoz
Rol: 202373524-8

Segundo semestre 2026

Índice

1. Introducción **2**

2. Preparación del entorno **3**

2.1. Instalación de Neo4j y carga de datos 3

2.2. Instalación y verificación GDS 4

2.3. Verificación de carga 4

3. Exploración y consultas Cypher **5**

3.1. Visualización e interpretación del grafo 5

3.2. Casa con más visitas 6

3.3. Estudiante con más dulces 7

3.4. Visitas sin dulces 8

1. Introducción

En este informe se repite el caso de Voice Managers de la tarea anterior, pero ahora sobre Apache Cassandra. La diferencia principal con MongoDB es que en Cassandra las tablas se diseñan a partir de las consultas que va a hacer la aplicación.

Se trabajó con un clúster de tres nodos levantado con Docker, un keyspace con factor de replicación 3 y tres tablas pensadas para las consultas pedidas. Sobre eso se analizaron las consultas con `TRACING`, se probó `ALLOW FILTERING`, se crearon índices SAI y se comparó la desnormalización con el uso de índices.

Los archivos de la tarea (`schema.cql`, el script de generación de datos, las consultas y las capturas) están en el repositorio público `https://github.com/Sxbayb/Tareas-BDDA`.

2. Preparación del entorno

2.1. Instalación de Neo4j y carga de datos

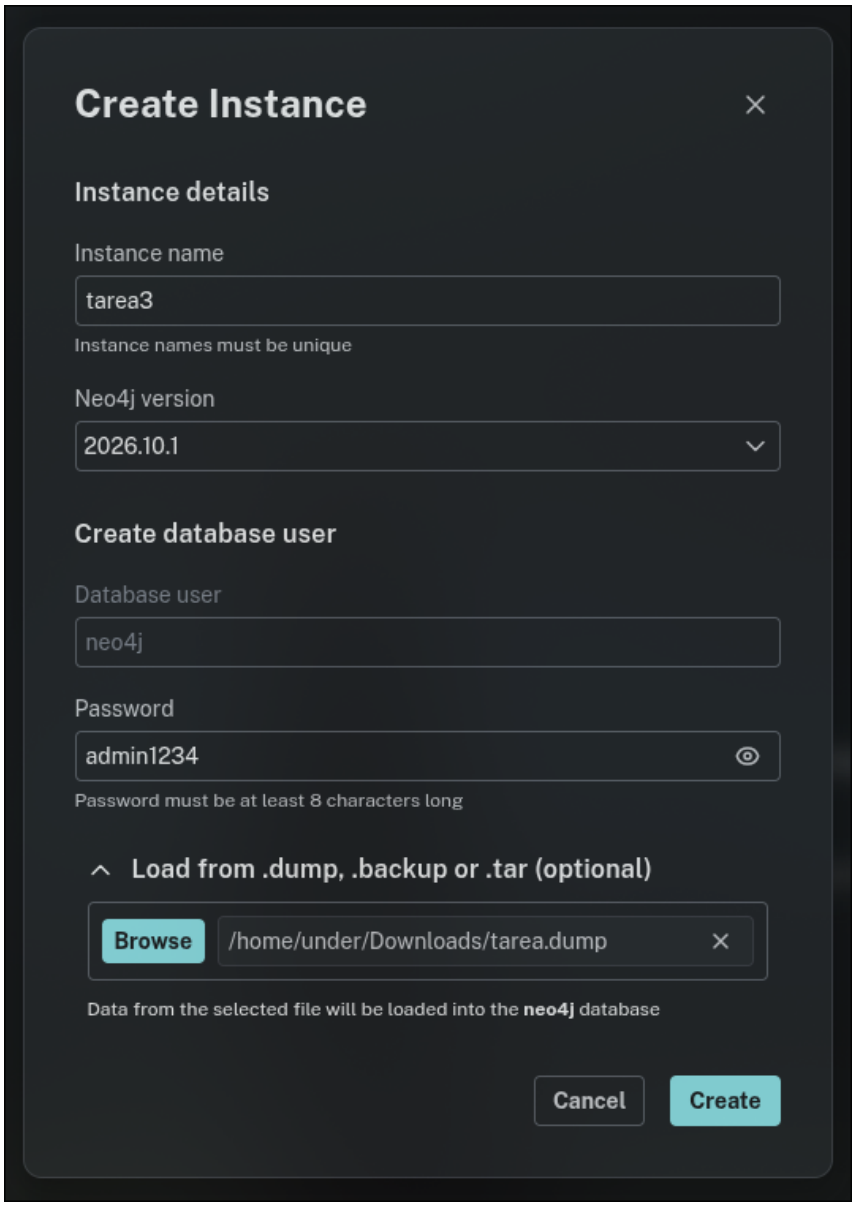


Figura 1: Creación de la instancia y carga de archivo dump.

2.2. Instalación y verificación GDS

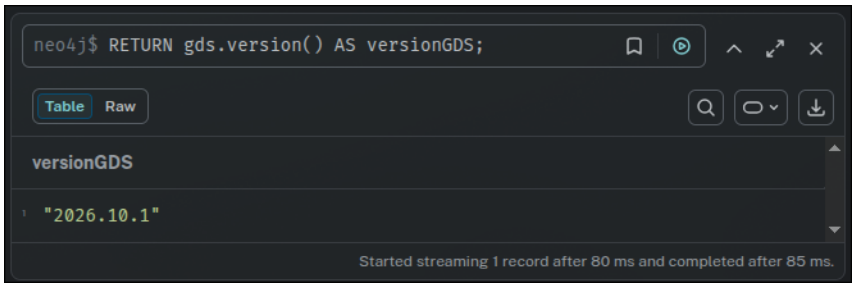


Figura 2: Descarga y comprobación de versión de GDS.

2.3. Verificación de carga



Figura 3: Descarga y comprobación de versión de GDS.

3. Exploración y consultas Cypher

3.1. Visualización e interpretación del grafo

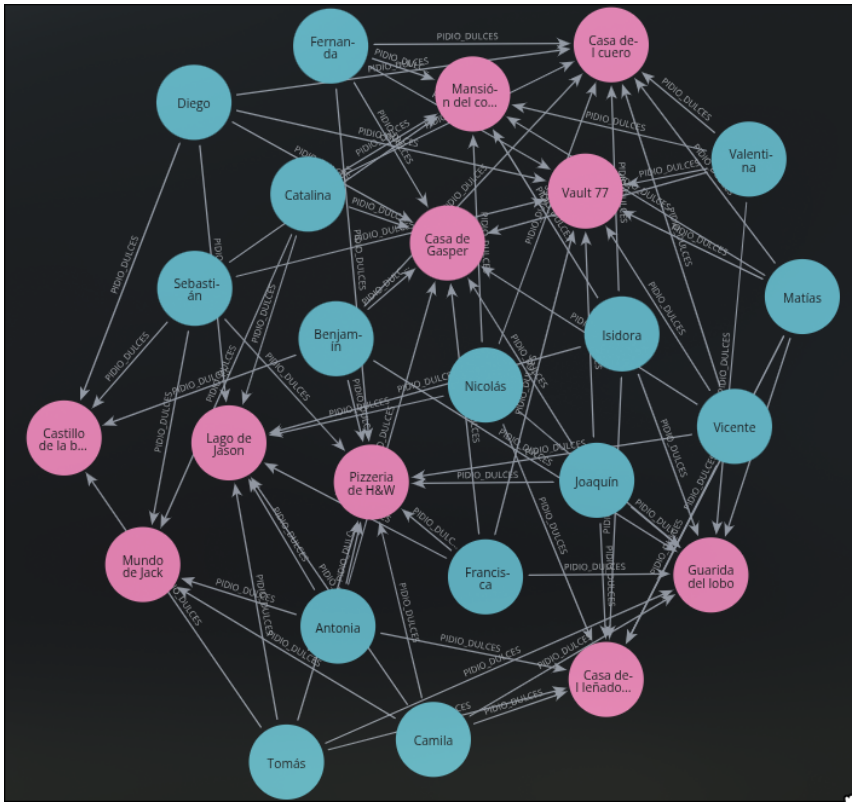


Figura 4: Descarga y comprobación de versión de GDS.

En el grafo presentado se pueden observar dos tipos de nodos. El primer tipo de nodo que se puede ver son los **nodos de estudiantes**(nodos azules) los cuales representan a cada uno de los 15 estudiantes del curso que salieron a pedir dulces. Su propiedad principal es `id`, la cual almacena como `strings` el nombre del estudiante. El otro tipo de nodo que se puede ver son los **nodos de las casas**(nodos rosados) los cuales representan cada una de las 10 casas disponibles para pedir dulces. Posee un `id` numérico interno y una propiedad `direccion` que contiene su nombre temático.

Se puede apreciar en el grafo como se tienen relaciones dirigidas, el sentido que se puede apreciar es que la dirección va siempre desde un **nodo estudiante** a un **nodo casa**. Cada relación refleja la acción física y semántica de la actividad: es el estudiante quien actúa como emisor u origen de la visita, desplazándose hacia la casa como lugar de destino. Una arista representa el hecho de que ese estudiante visitó esa casa específica, independientemente de siempre recibió o no dulces.

Cada relación `PIDIO_DULCES` almacena dos propiedades:

- `cantidad`: Representa el número de dulces obtenidos en esa visita puntual. Puede ser 0 (no recibe dulces), 1 (recibió un dulce) o 2 (recibió dos dulces).
- `tipo`: Registra la variedad del dulce recibido, toma valor `'Ninguno'` en caso de que la `cantidad` sea 0.

En teoría de grafos, un grafo es bipartito si un conjunto de vértices V puede dividirse en dos subconjuntos disjuntos e independientes, tales que toda arista de E conecta un vértice de V_1 con uno de V_2. En este modelo es posible apreciar como tenemos todos los nodos particionados

en dos clases independientes: $V_1 =$ Estudiantes y $V_2 =$ Casas. Además, todas las relaciones van exclusivamente entre un elemento de V_1 y un elemento de V_2. Por último, no hay ninguna relación que conecte a un estudiante con otro estudiante, ni existe ninguna relación que conecte una casa con otra casa.

Por estas razones es posible decir que el grafo con el que se está trabajando es un grafo bipartito.

3.2. Casa con más visitas



Figura 5: Ranking de las 10 casas con más visitas.



Figura 6: Empate en casas con más visitas.

Para este contexto se debe considerar una visita que tenga `cantidad=0` ya que la existencia de la relación dirigida representa el hecho de que un estudiante se trasladó y visitó esa casa puntual durante Halloween. La relación existe independiente del resultado de la interacción. El hecho de que la propiedad `cantidad` tome un valor de 0 modela una visita infructuosa (por ejemplo, nadie atendió la puerta o ya no habían dulces), pero la casa efectivamente fue visitada por el estudiante. Como se pide medir la afluencia de visitas recibidas por la casa, sería un error descartar la arista o solo sumar los dulces entregados por una casa, ya que eso mediría el inventario de dulces entregadas y no el tráfico de personas que acudieron al lugar.

3.3. Estudiante con más dulces

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
RETURN e.id AS estudiante,
       count(r) AS visitas,
       sum(r.cantidad) AS totalDulces
ORDER BY totalDulces DESC, estudiante ASC;
```

Table Raw

estudiante	visitas	totalDulces
"Tomás"	5	7
"Camila"	5	6
"Joaquin"	5	6
"Matias"	5	6
"Benjamin"	5	5
"Catalina"	5	5
"Fernanda"	5	5
"Francisca"	5	5
"Isidora"	5	5
"Nicolás"	5	5

Started streaming 15 records after 398 ms and completed after 414 ms.

Figura 7: Ranking de los 10 estudiantes con más dulces.

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
WITH e.id AS estudiante, count(r) AS visitas, sum(r.cantidad) AS totalDulces

WITH collect({estudiante: estudiante, visitas: visitas, totalDulces: totalDulces}) AS ranking,
     max(totalDulces) AS maxDulces

UNWIND ranking AS est
WITH est, maxDulces
WHERE est.totalDulces = maxDulces
RETURN est.estudiante AS ganador,
       est.visitas AS visitas,
       est.totalDulces AS dulcesObtenidos;
```

Table Raw

ganador	visitas	dulcesObtenidos
"Tomás"	5	7

Started streaming 1 record after 357 ms and completed after 366 ms.

Figura 8: Estudiante con más dulces.

Contar las visitas es una métrica estructural que mide el grado de salida del estudiante en el grafo, reflejando únicamente la cantidad de aristas o conexiones que posee. En cambio, sumar dulces cuantifica el valor acumulado de una propiedad numérica propia de cada relación. Dado que cada visita puede otorgar 0, 1 y 2 dulces, contar aristas solo indica el número de lugares concurridos, más no el beneficio o recompensa total obtenida.

El modelo fija como regla que cada estudiante visita exactamente 5 casas distintas, por lo que el conteo estructural de visitas es idéntico para todos. No obstante, el resultado de cada visita es independiente y asinga valores de 0, 1 y 2 dulces de forma estocástica. Debido a esta variabilidad en la propiedad `cantidad`, la suma total de cada alumno puede variar dentro del rango de 0 a 10 dulces, produciendo totales finales distintos a pesar de tener la misma cantidad de visitas.

3.4. Visitas sin dulces

```
MATCH (e:Estudiante)-[r:PIDIO_DULCES]->(c:Casa)
WHERE r.cantidad = 0
RETURN e.id AS estudiante,
       c.direccion AS direccionCasa,
       r.tipo AS tipo
ORDER BY estudiante ASC, direccionCasa ASC;
```

Table Raw

estudiante	direccionCasa	tipo
"Antonia"	"Mundo de Jack"	"Ninguno"
"Benjamín"	"Castillo de la bestia"	"Ninguno"
"Catalina"	"Lago de Jason"	"Ninguno"
"Diego"	"Casa del cuero"	"Ninguno"
"Fernanda"	"Vault 77"	"Ninguno"
"Francisca"	"Casa de Gasper"	"Ninguno"
"Nicolás"	"Lago de Jason"	"Ninguno"
"Sebastián"	"Castillo de la bestia"	"Ninguno"
"Sebastián"	"Pizzeria de H6W"	"Ninguno"
"Valentina"	"Mansión del conde"	"Ninguno"

Started streaming 12 records after 505 ms and completed after 518 ms.

Figura 9: Visitas sin dulces.

```
MATCH (:Estudiante)-[r:PIDIO_DULCES]->(:Casa)
WITH count(r) AS totalVisitas,
     count(CASE WHEN r.cantidad = 0 THEN 1 END) AS visitasSinDulces
RETURN visitasSinDulces,
       totalVisitas,
       round((visitasSinDulces * 100.0) / totalVisitas, 2) AS porcentajeSinDulces;
```

Table Raw

visitasSinDulces	totalVisitas	porcentajeSinDulces
12	75	16.0

Started streaming 1 record after 379 ms and completed after 410 ms.

Figura 10: Porcentaje de visitas sin dulces.

Durante la generación de la base de datos, la asignación de dulces en cada visita se modeló como un experimento estocástico independiente con una probabilidad teórica del 20 % para el resultado `cantidad=0`. La discrepancia entre el porcentaje real observado en la consulta (16 %) y dicho 20 % se fundamenta en principios estadísticos y en el tamaño de la muestra:

- **Muestra finita y variabilidad muestral:** La base registra un universo total de exactamente 75 visitas. Según la ley de los grandes números, la frecuencia relativa observada de un evento converge a su probabilidad teórica únicamente cuando el número de repeticiones tiende a infinito. En un conjunto acotado de 75 eventos, el 20 % representa una esperanza matemática de $75 \times 0.20 = 15$ visitas teóricas, más no una cota determinista o forzada.
- **Independencia de los eventos generados:** Cada visita fue asignada de forma pseudoaleatoria e independiente. El algoritmo no aplicó cuotas rígidas para asegurar que exactamente 15 de las 75 relaciones tuvieran valor cero, por lo que cualquier fluctuación observada en torno a la media esperada constituye una desviación muestral natural y propia de un proceso probabilístico discreto.